Übungsblatt 3 – Lösungen

Mathematik in Lean

Teil A: Auf Papier

Aufgabe A1 (Mengen sind Prädikate)

Es seien α ein Typ und s, t, u Teilmengen von α, in Lean `s t u : Set α`. Wie in der Vorlesung bedeutet $x \in s \cap t$ dasselbe wie $x \in s \wedge x \in t$, $x \in s \cup t$ dasselbe wie $x \in s \vee x \in t$ und $x \in s^{\mathrm{c}}$ dasselbe wie $\neg(x \in s)$. Ferner heißt $s \subseteq t$, dass aus $x \in s$ stets $x \in t$ folgt, und zwei Mengen sind gleich, wenn sie dieselben Elemente haben. Beweisen Sie die folgenden Aussagen, indem Sie sie für ein festes x auf Aussagenlogik zurückführen, und nennen Sie jeweils das aussagenlogische Gesetz, das dabei herauskommt.

(a) $s \cap (t \cup u) = (s \cap t) \cup (s \cap u)$.

(b) Aus $s \subseteq t$ folgt $t^{\mathrm{c}} \subseteq s^{\mathrm{c}}$.

(c) $(s \cap t)^{\mathrm{c}} = s^{\mathrm{c}} \cup t^{\mathrm{c}}$. Beweisen Sie die beiden Inklusionen einzeln. Eine davon braucht den Satz vom ausgeschlossenen Dritten, die andere nicht – welche ist welche? Braucht ihn die Regel $(s \cup t)^{\mathrm{c}} = s^{\mathrm{c}} \cap t^{\mathrm{c}}$ aus der Vorlesung?

Lösung. Für ein festes x stehen P, Q, R für $x \in s$, $x \in t$, $x \in u$. Bei den Gleichungen genügt es nach der Extensionalität, für jedes x die Aussagen links und rechts als äquivalent nachzuweisen.

(a) Links steht $P \wedge (Q \vee R)$, rechts $(P \wedge Q) \vee (P \wedge R)$ – das ist das Distributivgesetz. „$\Rightarrow$“: Aus $P \wedge (Q \vee R)$ erhält man P und $Q \vee R$. Im Fall Q gilt $P \wedge Q$, also die linke Seite der Disjunktion, im Fall R gilt $P \wedge R$, die rechte. „$\Leftarrow$“: Im Fall $P \wedge Q$ hat man P, und aus Q folgt $Q \vee R$; der Fall $P \wedge R$ geht genauso. Benutzt werden nur Einführung und Beseitigung von $\wedge$ und $\vee$ – der Beweis ist konstruktiv.

(b) Sei $x \in t^{\mathrm{c}}$, also $\neg Q$. Zu zeigen ist $\neg P$, also $P \to \mathrm{False}$. Gelte also P. Wegen $s \subseteq t$ folgt Q, und $\neg Q$, angewandt auf diesen Beweis von Q, liefert False. Das ist die Kontraposition $(P \to Q) \to (\neg Q \to \neg P)$. Als Term ist sie eine Komposition: Aus $h : P \to Q$ und $n : \neg Q$ wird $\lambda p.\, n\,(h\,p)$.

(c) Links steht $\neg(P \wedge Q)$, rechts $\neg P \vee \neg Q$ – das ist die De Morgansche Regel $\neg(P \wedge Q) \leftrightarrow \neg P \vee \neg Q$, hier in zwei Implikationen zerlegt.

„$\supseteq$“: Sei $\neg P \vee \neg Q$; zu zeigen ist $\neg(P \wedge Q)$. Gelte also $P \wedge Q$. Im Fall $\neg P$ widerspricht das P, im Fall $\neg Q$ widerspricht es Q. Diese Richtung ist konstruktiv.

„$\subseteq$“: Sei $\neg(P \wedge Q)$; zu zeigen ist $\neg P \vee \neg Q$. Nach dem Satz vom ausgeschlossenen Dritten gilt $P \vee \neg P$. Im Fall $\neg P$ ist die linke Seite bewiesen. Im Fall P gilt $\neg Q$ – aus Q folgte $P \wedge Q$, im Widerspruch zur Annahme –, also die rechte Seite.

Diese Richtung braucht den Satz vom ausgeschlossenen Dritten. Ein konstruktiver Beweis von $\neg P \vee \neg Q$ müsste angeben, *welche* Seite gilt, und das lässt sich aus $\neg(P \wedge Q)$ allein

nicht ablesen. Intuitionistisch ist $\neg(P \wedge Q) \to \neg P \vee \neg Q$ tatsächlich nicht beweisbar. Genau genommen genügt schon die schwächere Form $\neg P \vee \neg\neg P$: Im Fall $\neg\neg P$ gilt wieder $\neg Q$, denn aus Q folgte mit der Annahme $\neg P$, im Widerspruch zu $\neg\neg P$. Ohne jedes klassische Prinzip geht es aber nicht.

Die Regel aus der Vorlesung, $\neg(P \vee Q) \leftrightarrow \neg P \wedge \neg Q$, kommt in beiden Richtungen ohne ihn aus: Aus $\neg(P \vee Q)$ folgt $\neg P$, denn P ergäbe $P \vee Q$, und ebenso $\neg Q$. Umgekehrt führt $P \vee Q$ zusammen mit $\neg P \wedge \neg Q$ in beiden Fällen zum Widerspruch.

Aufgabe A2 (Rechnen mit Matrizen)

Es seien

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

(a) Berechnen Sie AB, BA, $\det A$, $\det B$ und $\det(AB)$. Ist das Matrixprodukt kommutativ? Gilt $\det(AB) = \det A \cdot \det B$?

(b) Sei $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ eine reelle Matrix mit $\det M = ad - bc \neq 0$. Zeigen Sie, dass $N = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$ die Gleichung $MN = I$ erfüllt; I ist die Einheitsmatrix.

(c) Zeigen Sie für jede reelle 2×2-Matrix M, alle $v, w \in \mathbb{R}^2$ und alle $c \in \mathbb{R}$: $M(v+w) = Mv + Mw$ und $M(cv) = c\,(Mv)$. Die Abbildung $v \mapsto Mv$ ist also linear.

(d) R dreht die Ebene um 90°. Berechnen Sie R^2 und R^4 und deuten Sie die Ergebnisse.

Hinweis: In (b) erst $M \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$ ausrechnen. In (c) genügt eine Komponente: Die i-te Komponente von Mv ist $M_{i1}v_1 + M_{i2}v_2$.

Lösung.

(a) Zeile mal Spalte:

$$AB = \begin{pmatrix} 1\cdot0+2\cdot1 & 1\cdot1+2\cdot0 \\ 3\cdot0+4\cdot1 & 3\cdot1+4\cdot0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}, \qquad BA = \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}.$$

Also ist $AB \neq BA$: Das Matrixprodukt ist nicht kommutativ. Von rechts multipliziert vertauscht B die Spalten, von links die Zeilen. Weiter ist $\det A = 1\cdot4 - 2\cdot3 = -2$, $\det B = 0\cdot0 - 1\cdot1 = -1$ und $\det(AB) = 2\cdot3 - 1\cdot4 = 2 = (-2)\cdot(-1)$, wie es $\det(AB) = \det A \cdot \det B$ verlangt.

(b) Es ist

$$M \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} = \begin{pmatrix} ad-bc & -ab+ba \\ cd-dc & -cb+da \end{pmatrix} = (ad-bc)\,I.$$

Weil $ad - bc \neq 0$ ist, darf man durch $ad - bc$ teilen: $MN = I$. Für $\det M = 0$ gibt es keine Inverse, denn aus $MN = I$ folgte $\det M \cdot \det N = \det I = 1$.

(c) Die i-te Komponente von $M(v + w)$ ist

$$M_{i1}(v_1 + w_1) + M_{i2}(v_2 + w_2) = (M_{i1}v_1 + M_{i2}v_2) + (M_{i1}w_1 + M_{i2}w_2),$$

also die i-te Komponente von $Mv + Mw$; das ist das Distributivgesetz in $\mathbb{R}$. Ebenso ist $M_{i1}(cv_1) + M_{i2}(cv_2) = c\,(M_{i1}v_1 + M_{i2}v_2)$. Zwei Vektoren sind gleich, wenn alle ihre Komponenten gleich sind.

(d) $R^2 = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix} = -I$: Zweimal um 90° ist die Drehung um 180°, also $v \mapsto -v$. Damit ist $R^4 = (-I)^2 = I$: Nach vier Vierteldrehungen steht jeder Vektor wieder an seinem Platz.

Aufgabe A3 (Lineare Abbildungen und ihr Kern)

Sei $f : \mathbb{R}^n \to \mathbb{R}^m$ linear, also $f(v+w) = f(v) + f(w)$ und $f(cv) = c\,f(v)$ für alle $v, w \in \mathbb{R}^n$ und alle $c \in \mathbb{R}$.

(a) Zeigen Sie $f(0) = 0$ und $f(-v) = -f(v)$ für alle $v \in \mathbb{R}^n$.

(b) Zeigen Sie, dass der Kern $\ker f = \{v \in \mathbb{R}^n \mid f(v) = 0\}$ ein Unterraum von $\mathbb{R}^n$ ist: Er enthält 0 und ist abgeschlossen unter Summen und Vielfachen.

(c) Bestimmen Sie Kern und Bild der Projektion $p : \mathbb{R}^2 \to \mathbb{R}^2$, $p(x, y) = (x, 0)$, und prüfen Sie die Dimensionsformel $\dim \ker p + \dim \operatorname{Bild} p = 2$.

(d) Die Verschiebung $g : \mathbb{R} \to \mathbb{R}$, $g(x) = x + 1$, ist nicht linear. Welche Aussage aus (a) verletzt sie? Ist $\{x \in \mathbb{R} \mid g(x) = 0\}$ ein Unterraum von $\mathbb{R}$?

Hinweis: Für $f(0) = 0$: Es ist $f(0) = f(0+0) = f(0) + f(0)$; ziehen Sie auf beiden Seiten $f(0)$ ab. Für $f(-v)$: $-v = (-1)\,v$.

Lösung.

(a) Es ist $f(0) + f(0) = f(0+0) = f(0) = f(0) + 0$. Kürzen, also auf beiden Seiten $f(0)$ abziehen, gibt $f(0) = 0$. Und $f(-v) = f\big((-1)\,v\big) = (-1)\,f(v) = -f(v)$.

(b) Nach (a) ist $0 \in \ker f$. Sind $v, w \in \ker f$, so ist $f(v+w) = f(v) + f(w) = 0 + 0 = 0$, also $v + w \in \ker f$. Ist $v \in \ker f$ und $c \in \mathbb{R}$, so ist $f(cv) = c\,f(v) = c \cdot 0 = 0$, also $cv \in \ker f$.

(c) $p(x, y) = (0, 0)$ gilt genau dann, wenn $x = 0$ ist. Also ist $\ker p = \{(0, y) \mid y \in \mathbb{R}\}$, die y-Achse, aufgespannt von $e_2 = (0, 1)$, und $\operatorname{Bild} p = \{(x, 0) \mid x \in \mathbb{R}\}$, die x-Achse, aufgespannt von $e_1 = (1, 0)$. Beide haben die Dimension 1, und $1 + 1 = 2 = \dim \mathbb{R}^2$.

(d) $g(0) = 1 \neq 0$, im Widerspruch zu (a). Schon die Additivität ist verletzt: $g(0+0) = 1$, aber $g(0) + g(0) = 2$. Die Menge $\{x \mid g(x) = 0\} = \{-1\}$ enthält die 0 nicht und ist deshalb kein Unterraum.

Aufgabe A4 (Induktion)

Zeigen Sie durch vollständige Induktion, dass für alle $n \in \mathbb{N}$ gilt:

(a) $4 \sum_{i=0}^{n} i^3 = n^2(n+1)^2$. Machen Sie vorher die Probe für $n = 3$.

(b) $n + 1 \leq 2^n$.

Lösung.

(a) Probe: $4 \cdot (0+1+8+27) = 4 \cdot 36 = 144 = 9 \cdot 16$. Induktionsanfang $n = 0$: $4 \cdot 0 = 0 = 0^2 \cdot 1^2$. Induktionsschritt: Gilt die Formel für n, so ist

$$\begin{aligned} 4 \sum_{i=0}^{n+1} i^3 &= 4 \sum_{i=0}^{n} i^3 + 4(n+1)^3 = n^2(n+1)^2 + 4(n+1)^3 \\ &= (n+1)^2(n^2 + 4n + 4) = (n+1)^2(n+2)^2. \end{aligned}$$

Zusammen mit der Gaußschen Summenformel $2\sum_{i=0}^{n} i = n(n+1)$ heißt das

$$\sum_{i=0}^{n} i^3 = \Big(\sum_{i=0}^{n} i\Big)^2.$$

(b) Induktionsanfang: $0+1 = 1 = 2^0$. Induktionsschritt: Aus $n+1 \le 2^n$ folgt $2^{n+1} = 2 \cdot 2^n \ge 2(n+1) = (n+2) + n \ge n+2$. Mathlib hat den Satz als `Nat.lt_two_pow_self`, in der Form $n < 2^n$ – in $\mathbb{N}$ ist $n < m$ dasselbe wie $n+1 \le m$.

Aufgabe A5 (Was der Typcheck sieht)

(a) In Lean ist `Set` α definitorisch $\alpha \to \mathrm{Prop}$, und $x \in s$ ist die Anwendung $s\,x$ (Abschnitt 3.6, „Mengen sind Prädikate“). Sei α ein Typ, der als Variable im Kontext steht. Zeigen Sie mit der Regel (App) aus Abschnitt 3.5, dass der Ausdruck $x \in x$ – und damit auch $x \notin x$ – weder im Kontext $x : \alpha$ noch im Kontext $x : \mathrm{Set}\,\alpha$ einen Typ hat. Welcher Term des λ-Kalküls aus Vorlesung 1 scheitert an derselben Stelle?

(b) Einsetzen: Im Kontext $x : \mathbb{N}$ seien $f = x + 2$ und $g = 3 \cdot x$. Berechnen Sie $g[f/x]$ und $f[g/x]$; dabei heißt $t[a/x]$: in t für x den Term a einsetzen. Sind die beiden Ergebnisse definitorisch gleich?

(c) Ist im Kontext $x : \mathbb{N}$ der Term $x + 0$ definitorisch gleich x? Und $0 + x$? Zur Erinnerung: Die Addition auf $\mathbb{N}$ rekursiert über das zweite Argument, $n+0 \equiv n$ und $n+(m+1) \equiv (n+m)+1$.

Hinweis: Eine Variable hat den Typ, unter dem sie im Kontext steht, und über (Conv) jeden dazu definitorisch gleichen. Definitorisch gleiche Terme haben dieselbe Normalform (Abschnitt 3.5).

Lösung.

(a) $x \in x$ steht für die Anwendung $x\,x$, und eine Anwendung bekommt ihren Typ nur über die Regel

$$\frac{\Gamma \vdash x : (y : A) \to B \qquad \Gamma \vdash x : A}{\Gamma \vdash x\,x : B[x/y]}\ (\textsc{App})$$

(dazu höchstens noch (Conv)). Die Typen von x sind genau die, die zum Typ im Kontext definitorisch gleich sind.

Im Kontext $x : \alpha$ müsste $\alpha \equiv (y : A) \to B$ sein. Auf die Variable α passt aber keine Reduktionsregel; sie ist ihre eigene Normalform und kein Funktionstyp. Weil definitorisch gleiche Terme dieselbe Normalform haben, ist das unmöglich: x ist keine Funktion.

Im Kontext $x : \mathrm{Set}\,\alpha$ ist x eine Funktion, denn $\mathrm{Set}\,\alpha \equiv \alpha \to \mathrm{Prop}$; also ist $A \equiv \alpha$. Die zweite Prämisse verlangt dann $x : \alpha$, also $\alpha \to \mathrm{Prop} \equiv \alpha$ – ein Funktionstyp und eine Variable, zwei verschiedene Normalformen. Auch das ist unmöglich.

In beiden Fällen bräuchte man einen Typ, der definitorisch sein eigener Prädikattyp ist: $\alpha \equiv \alpha \to \mathrm{Prop}$. (Lean scheitert sogar schon eine Stufe früher, bei der Notation: Für Elemente und Mengen desselben Typs findet es keine Elementbeziehung.)

Dasselbe Muster, eine Variable auf sich selbst angewandt, steckt in $\lambda x.\, x\,x$ und damit in $(\lambda x.\, x\,x)\,(\lambda x.\, x\,x)$, dem Term aus Vorlesung 1, der für immer zu sich selbst reduziert. Im einfach getypten λ-Kalkül bräuchte $x : A$ die Gleichung $A = A \to B$, und ein Typ kann nicht echt in sich selbst vorkommen. Russells Menge und der nicht abbrechende Term scheitern an derselben Stelle: an der Selbstanwendung.

(b) $g[f/x] = 3 \cdot (x + 2)$ und $f[g/x] = 3 \cdot x + 2$. Die beiden Terme sind nicht definitorisch gleich: Wären sie es, so auch nach dem Einsetzen von 0 für x – Rechnen und Einsetzen vertragen sich. Es ergeben sich aber $3 \cdot (0 + 2) \equiv 6$ und $3 \cdot 0 + 2 \equiv 2$, zwei verschiedene Normalformen. Beim Einsetzen kommt es also auf die Reihenfolge an.

(c) Nach der ersten Rechenregel ist $x + 0 \equiv x$. Bei $0 + x$ steht im zweiten Argument die Variable x; keine der beiden Regeln greift, der Term rechnet nicht weiter und ist nicht definitorisch gleich x – obwohl $0 + n = n$ für jedes n gilt, per Induktion (`Nat.zero_add`). Definitorisch gleich ist nur, was der Kern ausrechnen kann ($\equiv$), nicht, was sich beweisen lässt. B4 (d) zeigt den Unterschied in Lean.

Teil B: In Lean

Aufgabe B1 (Formalisierung von A1)

Die Aussagen aus A1 in Lean. Teil (b) soll ein Beweis im Termmodus werden, ohne `by`; Teil (c) ist in die beiden Inklusionen zerlegt.

Lean `lean/Blatt03.lean`

```
/-- Aufgabe B1 (a): Distributivitaet. -/
theorem distributiv {α : Type*} (s t u : Set α) :
    s ∩ (t ∪ u) = (s ∩ t) ∪ (s ∩ u) := by
  -- ‘ext x‘, ‘constructor‘, dann zerlegen, etwa mit ‘rintro ⟨hs, ht | hu⟩‘
  sorry

/-- Aufgabe B1 (b), im Termmodus: Komplementbildung kehrt Inklusionen um. -/
theorem kompl_antiton {α : Type*} (s t : Set α) (h : s ⊆ t) : tᶜ ⊆ sᶜ :=
  sorry

/-- Aufgabe B1 (c): De Morgan fuer den Durchschnitt, eine Inklusion nach der
anderen. Fuer eine der beiden brauchen Sie ‘Classical.em‘. -/
theorem de_morgan_1 {α : Type*} (s t : Set α) : sᶜ ∪ tᶜ ⊆ (s ∩ t)ᶜ := by
  sorry

theorem de_morgan_2 {α : Type*} (s t : Set α) : (s ∩ t)ᶜ ⊆ sᶜ ∪ tᶜ := by
  sorry

theorem de_morgan {α : Type*} (s t : Set α) : (s ∩ t)ᶜ = sᶜ ∪ tᶜ :=
  -- aus den beiden Inklusionen, mit ‘Set.Subset.antisymm‘
  sorry

/- Worauf beruhen Ihre beiden Beweise? Achten Sie auf ‘Classical.choice‘. -/
#print axioms de_morgan_1
#print axioms de_morgan_2
```

Hinweis: (a) Nach `ext x` ist eine Äquivalenz zu zeigen, `constructor` macht daraus zwei Implikationen. `rintro` zerlegt eine Annahme schon beim Einführen: ein $\wedge$ in seine beiden Teile, ein $\vee$ in zwei Fälle. Die Ziele setzen `Or.inl`, `Or.inr` und der anonyme Konstruktor $\langle \cdot, \cdot \rangle$ zusammen. (b) $x \in t^c$ ist definitorisch $x \in t \to \text{False}$. Ein Beweis von $t^c \subseteq s^c$ ist also eine Funktion, die ein x, einen Beweis von $x \in t^c$ und einen von $x \in s$ nimmt und False liefert. Ein Argument, das im Rumpf nicht vorkommt, schreibt man als `_`; sonst warnt Lean hier vor einer unbenutzten Variablen. (c) `Classical.em` liefert für jede Aussage P einen Beweis von $P \vee \neg P$, `rcases` zerlegt ihn in zwei Fälle. Die Ausgabe von `#print axioms` erscheint im Infofenster. Achten Sie dort auf `Classical.choice`: `Classical.em` ist in Lean ein Satz, und `#print axioms` nennt die Axiome, aus denen er bewiesen ist.

Lösung. (a) ist A1 (a) Schritt für Schritt: `rintro` zerlegt die Annahme in die Fälle, `Or.inl` und `Or.inr` wählen die Seite der Disjunktion.

(b) ist die Kontraposition als Komposition: `hx (h hs)` wendet erst `h`, dann `hx` an. Das x braucht man nicht zu benennen, Lean erschließt es aus `hs`; daher der Unterstrich.

(c) Die eine Inklusion ist eine Fallunterscheidung über die Annahme, für die andere liefert `Classical.em (x ∈ s)` die beiden Fälle aus A1 (c).

Was `#print axioms` meldet, betrifft die *Beweise*. Der Beweis von `de_morgan_2` beruht über `Classical.em` auf `propext`, `Classical.choice` und `Quot.sound`, der von `de_morgan_1` auf keinem Axiom. Entscheidend ist `Classical.choice`: Daraus ist `Classical.em` bewiesen (Diaconescu). `propext` und `Quot.sound` sind keine klassischen Prinzipien; sie kommen schon mit `ext` oder `simp` herein – beweist man `de_morgan_1` etwa mit `simp`, erscheint `propext`. Damit bestätigt Lean, dass `de_morgan_1` konstruktiv geht. Dass `de_morgan_2` ohne klassisches Axiom *nicht* geht, kann `#print axioms` nicht zeigen: Es meldet nur, was dieser eine Beweis benutzt. Das bleibt die Aussage aus A1 (c), ein Satz über die Logik.

Mit `ext x` und `simp` allein geht es übrigens weder bei (a) noch bei der Gleichung in (c): `simp` wendet das Distributivgesetz nicht von selbst an und formt $\neg(P \wedge Q)$ zu $P \to \neg Q$ um. In Mathlib stehen die drei Aussagen als `Set.inter_union_distrib_left`, `Set.compl_subset_compl` (dort als Äquivalenz) und `Set.compl_inter`.

Lean, Lösung `loesungen/Blatt03_Loesung.lean`

```
/-- Aufgabe B1 (a): Distributivitaet. -/
theorem distributiv {α : Type*} (s t u : Set α) :
    s ∩ (t ∪ u) = (s ∩ t) ∪ (s ∩ u) := by
  -- ‘ext x‘, ‘constructor‘, dann zerlegen, etwa mit ‘rintro ⟨hs, ht | hu⟩‘
  ext x
  constructor
  · rintro ⟨hs, ht | hu⟩
    · exact Or.inl ⟨hs, ht⟩
    · exact Or.inr ⟨hs, hu⟩
  · rintro (⟨hs, ht⟩ | ⟨hs, hu⟩)
    · exact ⟨hs, Or.inl ht⟩
    · exact ⟨hs, Or.inr hu⟩

/-- Aufgabe B1 (b), im Termmodus: Komplementbildung kehrt Inklusionen um. -/
theorem kompl_antiton {α : Type*} (s t : Set α) (h : s ⊆ t) : tᶜ ⊆ sᶜ :=
  fun _ hx hs => hx (h hs)

/-- Aufgabe B1 (c): De Morgan fuer den Durchschnitt, eine Inklusion nach der
anderen. Fuer eine der beiden brauchen Sie ‘Classical.em‘. -/
theorem de_morgan_1 {α : Type*} (s t : Set α) : sᶜ ∪ tᶜ ⊆ (s ∩ t)ᶜ := by
  rintro x (hs | ht) ⟨h1, h2⟩
  · exact hs h1
  · exact ht h2

theorem de_morgan_2 {α : Type*} (s t : Set α) : (s ∩ t)ᶜ ⊆ sᶜ ∪ tᶜ := by
  intro x hx
  rcases Classical.em (x ∈ s) with hs | hs
  · exact Or.inr (fun ht => hx ⟨hs, ht⟩)
  · exact Or.inl hs

theorem de_morgan {α : Type*} (s t : Set α) : (s ∩ t)ᶜ = sᶜ ∪ tᶜ :=
  -- aus den beiden Inklusionen, mit ‘Set.Subset.antisymm‘
  Set.Subset.antisymm (de_morgan_2 s t) (de_morgan_1 s t)

/- Worauf beruhen Ihre beiden Beweise? Achten Sie auf ‘Classical.choice‘. -/
#print axioms de_morgan_1
#print axioms de_morgan_2
```

Aufgabe B2 (Formalisierung von A2)

A2 in Lean. In $\mathbb{Z}$ rechnet der Kern Produkte, Determinanten und Potenzen selbst aus. Die Teile (b) und (c) gelten für alle reellen Matrizen; dort vergleicht man Einträge.

Lean lean/Blatt03.lean

```
/- ‘!![a, b; c, d]‘ ist eine 2×2-Matrix, ‘![x, y]‘ ein Vektor in ℝ²; nach
   ‘open Matrix‘ heisst Matrix mal Vektor ‘M *ᵥ v‘. In ℤ rechnet der Kern
   Produkte, Determinanten und Potenzen mit ‘decide‘ selbst aus. In ℝ geht
   das nicht; dort vergleicht man Eintraege. -/
open Matrix

/-- Aufgabe B2 (a): die Rechnungen aus A2 (a). -/
example : !![1, 2; 3, 4] * !![0, 1; 1, 0] = (!![2, 1; 4, 3] : Matrix (Fin 2) (Fin
    2) ℤ) := by
  sorry

example : (!![0, 1; 1, 0] * !![1, 2; 3, 4] : Matrix (Fin 2) (Fin 2) ℤ) ≠
    !![1, 2; 3, 4] * !![0, 1; 1, 0] := by
  sorry

example : (!![1, 2; 3, 4] : Matrix (Fin 2) (Fin 2) ℤ).det = -2 := by
  sorry

/-- Aufgabe B2 (b): die Formel fuer die Inverse aus A2 (b). -/
theorem inverse_2x2 (a b c d : ℝ) (h : a * d - b * c ≠ 0) :
    !![a, b; c, d] * ((a * d - b * c)⁻¹ • !![d, -b; -c, a]) = 1 := by
  -- wie in A2 (b): erst ‘have‘ mit M * !![d, -b; -c, a] = (a * d - b * c) • 1
  sorry

/-- Aufgabe B2 (c): ‘v ↦ M v‘ ist linear (A2 (c)). -/
theorem mulVec_linear (M : Matrix (Fin 2) (Fin 2) ℝ) (v w : Fin 2 → ℝ) (c : ℝ) :
    M *ᵥ (v + w) = M *ᵥ v + M *ᵥ w ∧ M *ᵥ (c • v) = c • (M *ᵥ v) := by
  sorry

/-- Aufgabe B2 (d): die Drehung ‘R‘ aus A2 (d), viermal hintereinander. -/
example : (!![0, -1; 1, 0] : Matrix (Fin 2) (Fin 2) ℤ) ^ 4 = 1 := by
  sorry
```

Hinweis: (a) und (d): `decide`. (b) Halten Sie mit `have` die Gleichung $M \cdot \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} = (ad - bc) \cdot I$ fest: `Matrix.mul_fin_two` rechnet das Produkt aus, `ext i j` vergleicht die Einträge, `fin_cases` geht die Indizes durch, danach `simp` und `ring`. Den Rest erledigen `Matrix.mul_smul`, `smul_smul`, `inv_mul_cancel₀` und `one_smul`. (c) `constructor`, dann für jede Hälfte `ext i`, `simp` mit `Matrix.mulVec`, `dotProduct` und `Fin.sum_univ_two`, und `ring`.

Lösung. (a) und (d): `decide` lässt den Kern die Einträge ausrechnen und vergleichen. Das geht in $\mathbb{Z}$, weil dort jede Gleichung zwischen Zahlen entscheidbar ist; in $\mathbb{R}$ ginge es nicht.

(b) ist die Rechnung aus A2 (b): `hM` ist $M \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} = (ad - bc)\, I$. Danach zieht `Matrix.mul_smul` den Faktor $(ad - bc)^{-1}$ nach vorn, `smul_smul` fasst die beiden Faktoren zusammen, und `inv_mul_cancel₀ h` macht aus $(ad - bc)^{-1}(ad - bc)$ die 1 – hier wird $ad - bc \neq 0$ gebraucht.

(c) ist die Komponentenrechnung aus A2 (c): Nach `simp` steht in jeder Komponente eine Gleichung zwischen reellen Zahlen, die `ring` beweist.

In Mathlib heißen die Aussagen `Matrix.det_fin_two_of`, `Matrix.mulVec_add` und

`Matrix.mulVec_smul`.

Lean, Lösung loesungen/Blatt03_Loesung.lean

```
/- '!![a, b; c, d]' ist eine 2×2-Matrix, '![x, y]' ein Vektor in ℝ²; nach
   'open Matrix' heisst Matrix mal Vektor 'M *ᵥ v'. In ℤ rechnet der Kern
   Produkte, Determinanten und Potenzen mit 'decide' selbst aus. In ℝ geht
   das nicht; dort vergleicht man Eintraege. -/
open Matrix

/-- Aufgabe B2 (a): die Rechnungen aus A2 (a). -/
example : !![1, 2; 3, 4] * !![0, 1; 1, 0] = (!![2, 1; 4, 3] : Matrix (Fin 2)
    (Fin 2) ℤ) := by
  decide

example : (!![0, 1; 1, 0] * !![1, 2; 3, 4] : Matrix (Fin 2) (Fin 2) ℤ) ≠
    !![1, 2; 3, 4] * !![0, 1; 1, 0] := by
  decide

example : (!![1, 2; 3, 4] : Matrix (Fin 2) (Fin 2) ℤ).det = -2 := by
  decide

/-- Aufgabe B2 (b): die Formel fuer die Inverse aus A2 (b). -/
theorem inverse_2x2 (a b c d : ℝ) (h : a * d - b * c ≠ 0) :
    !![a, b; c, d] * ((a * d - b * c)⁻¹ • !![d, -b; -c, a]) = 1 := by
  -- wie in A2 (b): erst 'have' mit M * !![d, -b; -c, a] = (a * d - b * c) • 1
  have hM : !![a, b; c, d] * !![d, -b; -c, a] =
      (a * d - b * c) • (1 : Matrix (Fin 2) (Fin 2) ℝ) := by
    rw [Matrix.mul_fin_two]
    ext i j
    fin_cases i <;> fin_cases j <;> simp <;> ring
  rw [Matrix.mul_smul, hM, smul_smul, inv_mul_cancel₀ h, one_smul]

/-- Aufgabe B2 (c): 'v ↦ M v' ist linear (A2 (c)). -/
theorem mulVec_linear (M : Matrix (Fin 2) (Fin 2) ℝ) (v w : Fin 2 → ℝ) (c : ℝ)
    :
    M *ᵥ (v + w) = M *ᵥ v + M *ᵥ w ∧ M *ᵥ (c • v) = c • (M *ᵥ v) := by
  constructor
  · ext i
    simp [Matrix.mulVec, dotProduct, Fin.sum_univ_two]
    ring
  · ext i
    simp [Matrix.mulVec, dotProduct, Fin.sum_univ_two]
    ring

/-- Aufgabe B2 (d): die Drehung 'R' aus A2 (d), viermal hintereinander. -/
example : (!![0, -1; 1, 0] : Matrix (Fin 2) (Fin 2) ℤ) ^ 4 = 1 := by
  decide
```

Aufgabe B3 (Formalisierung von A3)

A3 in Lean. Die Abbildung f ist eine gewöhnliche Funktion von $\mathbb{R}^n$ nach $\mathbb{R}^m$, also von `Fin n` $\to \mathbb{R}$ nach `Fin m` $\to \mathbb{R}$; die Linearität steht als Voraussetzungen `hadd` und `hsmul` daneben. Den Kern bauen Sie gebündelt: als Element von `Submodule ℝ (Fin n → ℝ)`, also Trägermenge und drei Beweise. Teil (c) ist die Verschiebung aus A3 (d).

Lean lean/Blatt03.lean

```
/- In Mathlib waere ‘f‘ ein Element von ‘(Fin n → ℝ) →ₗ[ℝ] (Fin m → ℝ)‘. Hier
   ist ‘f‘ eine gewoehnliche Funktion von ℝ^n nach ℝ^m; die Linearitaet steht
   als Voraussetzungen ‘hadd‘ und ‘hsmul‘ daneben. -/

/-- Aufgabe B3 (a): ‘f‘ bildet ‘0‘ auf ‘0‘ ab ... -/
theorem lin_null {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hadd : ∀ v w, f (v + w) = f v + f w) : f 0 = 0 := by
  -- wie in A3 (a): erst ‘have‘ mit f 0 + f 0 = f 0 + 0, dann ‘add_left_cancel‘
  sorry

/-- ... und ‘-v‘ auf ‘-f v‘. -/
theorem lin_neg {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hsmul : ∀ (c : ℝ) v, f (c • v) = c • f v) (v : Fin n → ℝ) : f (-v) = -f v :=
    by
  sorry

/-- Aufgabe B3 (b): Der Kern als Unterraum, gebuendelt: Traegermenge und
drei Beweise. ‘v ∈ {v | f v = 0}‘ ist definitorisch ‘f v = 0‘;
‘rw [Set.mem_setOf_eq] at hv‘ macht das fuer ‘rw‘ sichtbar. -/
def kern {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hadd : ∀ v w, f (v + w) = f v + f w)
    (hsmul : ∀ (c : ℝ) v, f (c • v) = c • f v) : Submodule ℝ (Fin n → ℝ) where
  carrier := {v | f v = 0}
  zero_mem' := by
    sorry
  add_mem' := by
    sorry
  smul_mem' := by
    sorry

/-- Aufgabe B3 (c): Die Verschiebung aus A3 (d) ist nicht additiv. -/
theorem verschiebung_nicht_linear : ∃ g : ℝ → ℝ, ¬ ∀ x y, g (x + y) = g x + g y :=
  sorry
```

Hinweis: (a) Mit `hadd` rückwärts umgeschrieben, wird aus $f(0)+f(0)$ der Term $f(0+0)$; `add_left_cancel` kürzt. Für das Negative schreibt `neg_one_smul` den Vektor $-v$ als $(-1)\cdot v$. (b) Das Ziel $v \in \{v \mid f(v)=0\}$ ist definitorisch $f(v)=0$; `Set.mem_setOf_eq` macht das für `rw` sichtbar. Dazu `add_zero` und `smul_zero`. (c) Zeuge ist $x \mapsto x+1$ als `fun`-Ausdruck. Setzen Sie $x=y=0$ ein; `norm_num` erkennt $1=2$ als falsch.

Lösung. (a) Die Gleichung $f(0)+f(0)=f(0)+0$ beweist `rw [← hadd, add_zero, add_zero]`, und `add_left_cancel` kürzt. Für das Negative ist es die Rechnung $f(-v)=f((-1)\,v)=(-1)\,f(v)=-f(v)$, mit `neg_one_smul` in beiden Richtungen.

(b) Die drei Beweise sind A3 (b). `Set.mem_setOf_eq` macht aus $v \in \{v \mid f(v)=0\}$ die Gleichung $f(v)=0$, mit der `rw` arbeiten kann.

(c) Für $x=y=0$ verlangt die Additivität $(0+0)+1=(0+1)+(0+1)$, also $1=2$; `norm_num` macht daraus `False`.

Mathlib bündelt `f` mit `hadd` und `hsmul` zu einem Element von `(Fin n → ℝ) →ₗ[ℝ] (Fin m → ℝ)`; dessen Kern heißt `LinearMap.ker`.

Lean, Lösung loesungen/Blatt03_Loesung.lean

```
/- In Mathlib waere ‘f‘ ein Element von ‘(Fin n → ℝ) →ₗ[ℝ] (Fin m → ℝ)‘. Hier
   ist ‘f‘ eine gewoehnliche Funktion von ℝ^n nach ℝ^m; die Linearitaet steht
   als Voraussetzungen ‘hadd‘ und ‘hsmul‘ daneben. -/

/-- Aufgabe B3 (a): ‘f‘ bildet ‘0‘ auf ‘0‘ ab ... -/
theorem lin_null {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hadd : ∀ v w, f (v + w) = f v + f w) : f 0 = 0 := by
  -- wie in A3 (a): erst ‘have‘ mit f 0 + f 0 = f 0 + 0, dann ‘add_left_cancel‘
  have h : f 0 + f 0 = f 0 + 0 := by rw [← hadd, add_zero, add_zero]
  exact add_left_cancel h

/-- ... und ‘-v‘ auf ‘-f v‘. -/
theorem lin_neg {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hsmul : ∀ (c : ℝ) v, f (c • v) = c • f v) (v : Fin n → ℝ) : f (-v) = -f v
    := by
  rw [← neg_one_smul ℝ v, hsmul, neg_one_smul]

/-- Aufgabe B3 (b): Der Kern als Unterraum, gebuendelt: Traegermenge und
drei Beweise. ‘v ∈ {v | f v = 0}‘ ist definitorisch ‘f v = 0‘;
‘rw [Set.mem_setOf_eq] at hv‘ macht das fuer ‘rw‘ sichtbar. -/
def kern {n m : ℕ} (f : (Fin n → ℝ) → (Fin m → ℝ))
    (hadd : ∀ v w, f (v + w) = f v + f w)
    (hsmul : ∀ (c : ℝ) v, f (c • v) = c • f v) : Submodule ℝ (Fin n → ℝ) where
  carrier := {v | f v = 0}
  zero_mem' := by
    exact lin_null f hadd
  add_mem' := by
    intro v w hv hw
    rw [Set.mem_setOf_eq] at hv hw ⊢
    rw [hadd, hv, hw, add_zero]
  smul_mem' := by
    intro c v hv
    rw [Set.mem_setOf_eq] at hv ⊢
    rw [hsmul, hv, smul_zero]

/-- Aufgabe B3 (c): Die Verschiebung aus A3 (d) ist nicht additiv. -/
theorem verschiebung_nicht_linear : ∃ g : ℝ → ℝ, ¬ ∀ x y, g (x + y) = g x + g
    y :=
  ⟨fun x => x + 1, fun h => by have h0 := h 0 0; norm_num at h0⟩
```

Aufgabe B4 (Formalisierung von A4 und A5 (c))

(a) ist A4 (a), (b) die Probe für $n = 10$, (c) ist A4 (b), (d) stellt die Fragen aus A5 (c) als Gleichungen zwischen Funktionen. Das `=` in Lean ist die propositionale Gleichheit. Ob die Terme definitorisch gleich sind, wie A5 (c) fragt, zeigt nur `rfl`; wo es scheitert, hält das `#check_failure` fest.

Lean lean/Blatt03.lean

```
/-- Aufgabe B4 (a): die Summe der dritten Potenzen. -/
theorem summe_kuben (n : ℕ) :
    4 * (∑ i ∈ Finset.range (n + 1), i ^ 3) = n ^ 2 * (n + 1) ^ 2 := by
  sorry

/- Aufgabe B4 (b): die Probe fuer ‘n = 10‘. ‘#eval‘ rechnet nur aus; ein
   Beweis, den der Kern durch Rechnen prueft, steht darunter. -/
#eval 4 * (∑ i ∈ Finset.range 11, i ^ 3)
```

```
example : 4 * (∑ i ∈ Finset.range 11, i ^ 3) = 10 ^ 2 * 11 ^ 2 := by
  sorry

/-- Aufgabe B4 (c): die Ungleichung aus A4 (b). -/
theorem zweierpotenz (n : ℕ) : n + 1 ≤ 2 ^ n := by
  sorry

/- Aufgabe B4 (d), zu A5 (c). 'Nat.add' rekursiert ueber das zweite Argument.
   Versuchen Sie in beiden Faellen zuerst 'rfl'. -/
example : (fun x : ℕ => x + 0) = (fun x => x) :=
  sorry

example : (fun x : ℕ => 0 + x) = (fun x => x) :=
  sorry

/- Nicht definitorisch gleich: '#check_failure' erwartet, dass das scheitert. -/
#check_failure (rfl : (fun x : ℕ => 0 + x) = (fun x => x))
```

Hinweis: (a) geht wie die Summe der ungeraden Zahlen in der Vorlesung: `induction n with`, im Schritt `Finset.sum_range_succ` (letzten Summanden abspalten), wegen des Faktors 4 dazu `Nat.mul_add` (ausmultiplizieren), die Induktionsvoraussetzung `ih` und `ring`. (b) Lassen Sie den Kern rechnen: `decide`. (c) Wieder `induction n with`; im Schritt macht `pow_succ` aus 2^{k+1} das Produkt $2^k \cdot 2$, und danach genügt `omega` mit `ih`. (d) Wo `rfl` scheitert, hilft `funext` zusammen mit `Nat.zero_add`.

Lösung. (a) Nach `rw [Finset.sum_range_succ, Nat.mul_add, ih]` lautet das Ziel

$$k^2(k+1)^2 + 4(k+1)^3 = (k+1)^2(k+1+1)^2,$$

eine Gleichung im kommutativen Halbring $\mathbb{N}$, die `ring` erledigt.

(b) `decide` lässt den Kern beide Seiten ausrechnen, 12100. Das ist ein Beweis, aber nur für $n = 10$.

(c) Der Induktionsanfang $0 + 1 \le 2^0$ ist eine Rechnung für `simp`. Im Schritt lautet das Ziel nach `rw [pow_succ]`

$$k + 1 + 1 \le 2^k \cdot 2,$$

und die Induktionsvoraussetzung `ih` sagt $k + 1 \le 2^k$. Für `omega` ist 2^k eine Unbekannte wie jede andere; Ziel und Voraussetzung sind darin linear, und `omega` findet die Rechnung aus A4 (b), $2 \cdot 2^k \ge 2(k+1) \ge k + 2$, selbst.

(d) `rfl` gelingt für $x + 0$ und scheitert für $0 + x$ mit einem Typfehler: Der Kern kann $0 + x$ nicht zu x ausrechnen. `funext Nat.zero_add` beweist die Gleichung trotzdem, propositional, mit dem Induktionsbeweis `Nat.zero_add` an jeder Stelle x. Dass `rfl` hier scheitert, hält die Zeile mit `#check_failure` fest; so prüft der Compiler auch die negative Hälfte von A5 (c). Umgekehrt ginge die erste Gleichung auch propositional, mit `funext` und dem Satz $n + 0 = n$ – dass $x + 0$ und x definitorisch gleich sind, zeigt erst `rfl`. Genau nach diesem Unterschied fragt A5 (c).

Lean, Lösung loesungen/Blatt03_Loesung.lean

```
/-- Aufgabe B4 (a): die Summe der dritten Potenzen. -/
theorem summe_kuben (n : ℕ) :
    4 * (∑ i ∈ Finset.range (n + 1), i ^ 3) = n ^ 2 * (n + 1) ^ 2 := by
  induction n with
  | zero => simp
  | succ k ih =>
      rw [Finset.sum_range_succ, Nat.mul_add, ih]
      ring

/- Aufgabe B4 (b): die Probe fuer `n = 10`. `#eval` rechnet nur aus; ein
   Beweis, den der Kern durch Rechnen prueft, steht darunter. -/
#eval 4 * (∑ i ∈ Finset.range 11, i ^ 3)

example : 4 * (∑ i ∈ Finset.range 11, i ^ 3) = 10 ^ 2 * 11 ^ 2 := by
  decide

/-- Aufgabe B4 (c): die Ungleichung aus A4 (b). -/
theorem zweierpotenz (n : ℕ) : n + 1 ≤ 2 ^ n := by
  induction n with
  | zero => simp
  | succ k ih =>
      rw [pow_succ]
      omega

/- Aufgabe B4 (d), zu A5 (c). `Nat.add` rekursiert ueber das zweite Argument.
   Versuchen Sie in beiden Faellen zuerst `rfl`. -/
example : (fun x : ℕ => x + 0) = (fun x => x) :=
  rfl

example : (fun x : ℕ => 0 + x) = (fun x => x) :=
  funext Nat.zero_add

/- Nicht definitorisch gleich: `#check_failure` erwartet, dass das scheitert. -/
#check_failure (rfl : (fun x : ℕ => 0 + x) = (fun x => x))
```